

Coding Standards

Document ID: doc-004

Department: Engineering

Sensitivity: INTERNAL

Version 1.5 | Last updated: 2026-01-20 | Owner: Engineering Department

1. General Principles

- Code should be written for readability and maintainability first, performance second, unless the component is explicitly performance-critical.
- Prefer small, single-responsibility functions over large multi-purpose ones.
- Avoid premature abstraction; introduce shared abstractions only after a pattern repeats at least three times.

2. Naming Conventions

- Variables and functions: camelCase (TypeScript/JavaScript), snake_case (Python).
- Classes and types: PascalCase in both languages.
- Constants: UPPER_SNAKE_CASE.
- Names should describe intent, not implementation (e.g., isEligibleForDiscount, not checkFlag).

3. TypeScript / React

- Strict mode must be enabled in tsconfig.json; any is disallowed except with an inline justification comment.
- Components should be functional, using hooks; class components are deprecated in this codebase.
- Business logic should be extracted into hooks or utility modules, not embedded directly in JSX.
- All props must be explicitly typed via interfaces.

4. Python / FastAPI

- Follow PEP 8, enforced via ruff/flake8 in CI.
- All functions must have type hints; mypy runs in strict mode in CI.
- Business logic belongs in service modules, not in route handlers.
- Use Pydantic models for all request/response schemas.

5. Error Handling

Never silently swallow exceptions. Log with sufficient context (but never log secrets or PII). API errors must return structured error responses with a consistent shape (error_code, message, details).

6. Testing

- Minimum 80% line coverage for new code, enforced in CI.

- Unit tests must not depend on external services; use mocks/fakes.
- Integration tests are required for any change touching authentication, authorization, or payment logic.

7. Code Review

All PRs require at least one approval from a code owner of the affected module. Reviewers should check for correctness, security implications, test coverage, and adherence to these standards.

8. Security Baseline

- Never hardcode secrets, API keys, or credentials in source code.
- All user input must be validated at the API boundary.
- Dependencies are scanned automatically for known vulnerabilities on every PR; high/critical findings block merge.